FIAP • PÓS-GRADUAÇÃO • TESTES AUTOMATIZADOS

Ocean Cleaner

Evidências de Testes Automatizados

Suíte api-tests • BDD E2E • JSON Schema

Projeto ESG — ODS 14: Vida na Água

Testes de API ponta a ponta via Gateway, com ambiente completo em Docker Compose.

45	5	25	13
cenários Gherkin	features	endpoints	JSON Schemas

Integrantes: Gabriel Borges Cedraz de Santana • Matheus de Oliveira Radeze
Sabrina Pires Gomes da Silva • Herick Reis Nascimentos dos Santos

02 de novembro de 2026

Evidências geradas a partir do artifact e do log reais do GitHub Actions.

Sumário

Este documento registra as evidências da suíte de testes de API do Ocean Cleaner. A suíte foi implementada no diretório api-tests e executada no GitHub Actions contra o ambiente completo (Oracle, Eureka, Gateway e microsserviços). O foco aqui é a documentação e a comprovação de execução, não a reimplementação dos testes.

1. Contexto e objetivo . . . 4

2. Estratégia de testes e arquitetura . . . 5

2.1 Ferramentas . . . 5

2.2 Decisões de isolamento e dados . . . 5

3. Cenários Gherkin detalhados . . . 6

3.1 Infraestrutura e descoberta de serviços . . . 6

3.2 Gestão de áreas marítimas . . . 7

3.3 Gestão de voluntários . . . 9

3.4 Operações de limpeza e integração com voluntários . . . 11

3.5 Relatórios de coleta . . . 15

4. Cobertura das APIs e contratos . . . 17

4.1 Matriz de endpoints . . . 17

4.2 Contratos JSON Schema . . . 18

4.3 Comunicação entre serviços . . . 18

5. Configurações da suíte e do CI . . . 19

5.1 Dependências da suíte . . . 19

5.2 Script de execução . . . 19

5.3 Feature com o cenário de integração . . . 20

5.4 Contrato JSON Schema . . . 20

5.5 Job de CI . . . 22

5.6 Visão dos cenários no README da suíte . . . 22

6. Evidências de execução . . . 23

6.1 Resultado do CI . . . 23

6.2 Relatório Cucumber — visão geral . . . 24

6.3 Relatório Cucumber — features . . . 24

6.4 Relatório Cucumber — integração Feign . . . 26

6.5 Rastreabilidade . . . 26

7. Pipeline de CI . . . 27

7.1 Etapas . . . 27

7.2 Execução no PR . . . 27

8. Como reproduzir . . . 28

8.1 Subir o ambiente e executar . . . 28

8.2 Contra ambiente já no ar . . . 28
8.3 Filtrar por tag . . . 28
8.4 Relatórios . . . 28
9. Conclusão, checklist e limites . . . 29
9.1 Checklist da spec . . . 29
9.2 Limites e observações . . . 29
9.3 Conclusão . . . 29
Anexo A — Estrutura da suíte . . . 30
Anexo B — Índice de evidências . . . 30

1. Contexto e objetivo

O **Ocean Cleaner** organiza operações de limpeza de áreas marítimas. O domínio cobre áreas, operações, voluntários e relatórios de resíduos, alinhado ao pilar **Environmental** da agenda ESG (ODS 14) e ao pilar **Social** pelo voluntariado. A **Governança** é exercitada pela automação, pelos contratos e pela rastreabilidade no CI.

A suíte `api-tests` já está integrada ao projeto e é executada no pipeline. Ela adota BDD com Gherkin em português, Rest Assured para HTTP, Cucumber 7/JUnit 5 e JSON Schema para contrato. Este documento consolida os 45 cenários, as configurações e as evidências reais de execução.

Escopo quantitativo

Indicador	Resultado
Cenários Gherkin	45 cenários em 5 features, em português
Endpoints cobertos	25 endpoints: 22 de negócio + Gateway/Eureka
Contratos JSON Schema	13 schemas para entidades, listas, erros e infraestrutura
Execução de referência	GitHub Actions run 41266518830 (master), 45/45, 0 falhas
Tempo do job	4m18s no total; 90,93s de execução dos testes

Fonte da verdade

Cenários e schemas estão em `api-tests/src/test/resources/`. O relatório HTML, o JSON e o XML usados nas capturas foram baixados do artifact `api-tests-report` da execução 41266518830. O log do job foi obtido com `gh run view`.

Organização do documento

- Seção 2: estratégia, ferramentas e arquitetura E2E.
- Seção 3: descrição dos 45 cenários Gherkin, com tabela por feature e Gherkin completo.
- Seção 4: cobertura dos 25 endpoints e os 13 contratos.
- Seção 5: prints das configurações da suíte e do CI.
- Seção 6: evidências reais de execução, relatório HTML e resultado do CI.
- Seções 7 e 8: pipeline e comandos de reprodução.

2. Estratégia de testes e arquitetura

A suíte é **caixa-preta**: os testes não acessam o banco nem o código dos microsserviços. Cada requisição passa pelo Gateway, é roteada pelo Eureka e chega ao serviço correto; na conclusão de uma operação, o operacoes-ms chama o voluntarios-ms via Feign. O ambiente inteiro sobe com Docker Compose.

2.1 Ferramentas

Ferramenta	Versão	Papel
Java	21	Linguagem dos serviços e dos testes
Cucumber-JVM	7.20.1	Executa os cenários Gherkin
JUnit	5.11.4	Runner JUnit Platform Suite
Rest Assured	5.5.0	Cliente HTTP da suíte
JSON Schema Validator	5.5.0	Contratos das respostas
Docker Compose	ambiente completo	Oracle + Eureka + Gateway + MS

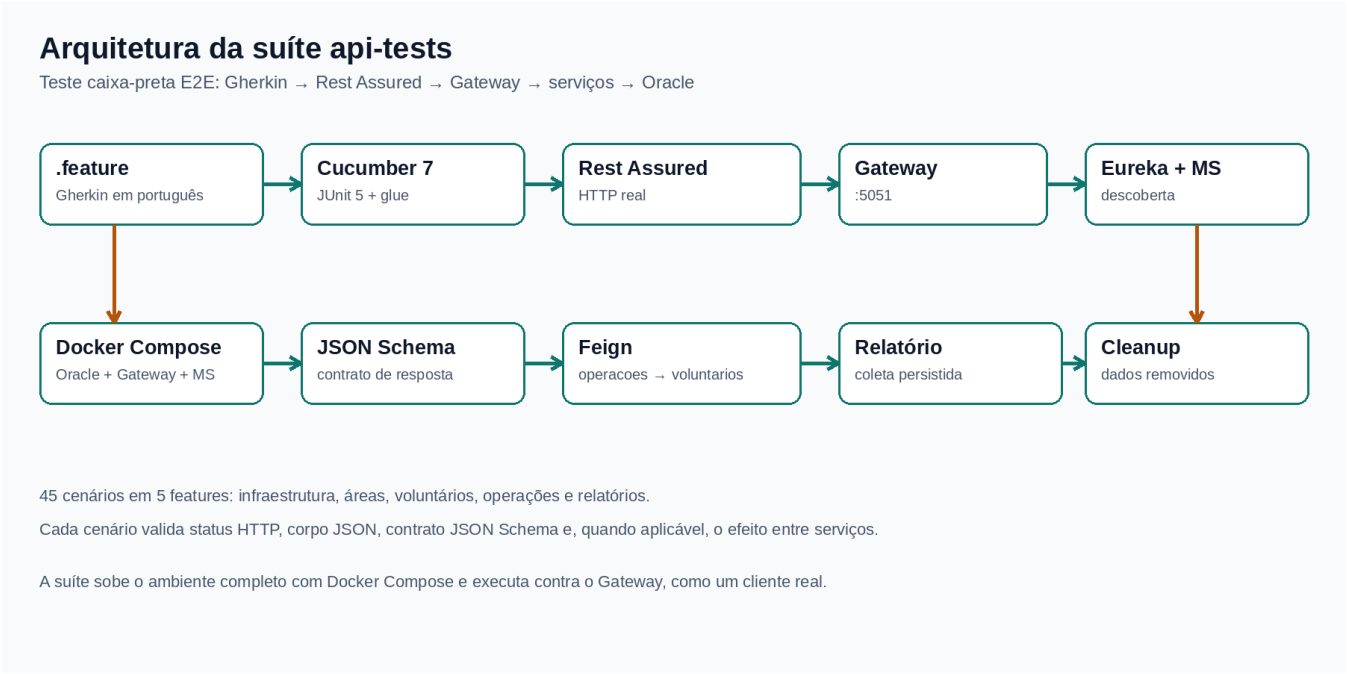


Figura 1 — Fluxo E2E da suíte api-tests: Gherkin, Rest Assured, Gateway, serviços e Oracle.

2.2 Decisões de isolamento e dados

- **Ambiente real:** a suíte usa o mesmo caminho do cliente (Gateway → Eureka → serviço), não chama controllers diretamente.
- **Dados únicos:** os cenários usam o sufixo {unico} para evitar colisão de nomes/e-mails.
- **Cleanup:** um hook remove os dados criados ao final de cada cenário, inclusive relatórios gerados pelo Feign.
- **Contratos:** cada resposta valida status, corpo JSON e JSON Schema; os schemas usam additionalProperties: false.
- **CI:** o job api-tests sobe o compose com senhas aleatórias, executa os cenários e derruba o ambiente com volume.

3. Cenários Gherkin detalhados

Os 45 cenários estão distribuídos em cinco features. Para cada feature são apresentados o resumo dos cenários expandidos (com resultado) e o Gherkin completo do arquivo, incluindo contexto, esboços de cenário e exemplos. Os cenários usam as tags `@principal`, `@integracao`, `@infraestrutura` e tags por serviço/recurso.

3.1 Infraestrutura e descoberta de serviços

Valida o Gateway e o Eureka: saúde/informações do ambiente, registro dos dois microsserviços e resposta 404 para rota sem serviço.

Cenários expandidos e resultado

#	Cenário	Tags	Resultado
1	Gateway saudável e identificando o ambiente	@infraestrutura	PASSOU
2	Microsserviços registrados no Eureka	@infraestrutura	PASSOU
3	Rota sem serviço correspondente no Gateway	@infraestrutura	PASSOU

Gherkin completo

```
# language: pt
@infraestrutura
Funcionalidade: Infraestrutura e descoberta de serviços
  Como equipe de operação do Ocean Cleaner
  Quero que o Gateway esteja saudável e os microsserviços registrados no Eureka
  Para que os clientes acessem toda a API por um único endereço
```

```
  Cenário: Gateway saudável e identificando o ambiente
    Quando envio um GET para "/actuator/health"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "health.json"
    E a resposta deve conter:
      | status | UP |
    Quando envio um GET para "/actuator/info"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "info.json"
    E a resposta deve conter:
      | app.name | Ocean Cleaner |
```

```
  Cenário: Microsserviços registrados no Eureka
    Quando consulto os serviços registrados no Eureka
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "eureka-apps.json"
    E o Eureka deve listar o serviço "GATEWAY"
    E o Eureka deve listar o serviço "OPERACOES-MS"
    E o Eureka deve listar o serviço "VOLUNTARIOS-MS"
```

```
  Cenário: Rota sem serviço correspondente no Gateway
    Quando envio um GET para "/servico-inexistente/recurso"
    Então o status da resposta deve ser 404
```

Destaques de comportamento

- Gateway responde health/info e o Eureka lista os microsserviços.
- Rota inexistente no Gateway retorna 404, comprovando o roteamento.

3.2 Gestão de áreas marítimas

Cobre CRUD de áreas, três validações de campos obrigatórios, autenticação, recurso inexistente e bloqueio de exclusão com operação vinculada.

Cenários expandidos e resultado

#	Cenário	Tags	Resultado
1	Cadastrar uma área marítima válida e consultá-la pelo id	@operacoes-ms @areas-maritimas @principal	PASSOU
2	Listar as áreas marítimas	@operacoes-ms @areas-maritimas	PASSOU
3	Atualizar o nível de poluição de uma área	@operacoes-ms @areas-maritimas	PASSOU
4	Remover uma área marítima sem operações	@operacoes-ms @areas-maritimas	PASSOU
5	Rejeitar cadastro sem o campo obrigatório "nome"	@operacoes-ms @areas-maritimas	PASSOU
6	Rejeitar cadastro sem o campo obrigatório "localizacao"	@operacoes-ms @areas-maritimas	PASSOU
7	Rejeitar cadastro sem o campo obrigatório "nivelPoluicao"	@operacoes-ms @areas-maritimas	PASSOU
8	Consultar uma área marítima inexistente	@operacoes-ms @areas-maritimas	PASSOU
9	Cadastrar área sem autenticação	@operacoes-ms @areas-maritimas	PASSOU
10	Impedir a remoção de uma área com operações vinculadas	@operacoes-ms @areas-maritimas	PASSOU

Gherkin completo

```
# language: pt
@operacoes-ms @areas-maritimas
Funcionalidade: Gestão de áreas marítimas
  Como coordenador de limpeza
  Quero cadastrar e manter as áreas marítimas poluídas
  Para planejar as operações de limpeza sobre elas

  Contexto:
    Dado que estou autenticado como administrador
```

```
  @principal
  Cenário: Cadastrar uma área marítima válida e consultá-la pelo id
    Quando envio um POST para "/operacoes-ms/areas-maritimas" com o corpo:
      """
      {"nome": "Praia do Gonzaga {unico}", "descricao": "Muito plástico na faixa de areia",
       "localizacao": "Santos - SP", "nivelPoluicao": "ALTO"}
      """
    Então o status da resposta deve ser 201
    E a resposta deve respeitar o contrato "area-maritima.json"
    E a resposta deve conter:
      | nome          | Praia do Gonzaga {unico} |
      | localizacao   | Santos - SP              |
      | nivelPoluicao | ALTO                     |
    E guardo o campo "id" da resposta como "areaId"
    Quando envio um GET para "/operacoes-ms/areas-maritimas/{areaId}"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "area-maritima.json"
    E a resposta deve conter:
      | id   | {areaId}                 |
      | nome | Praia do Gonzaga {unico} |
```

```
  Cenário: Listar as áreas marítimas
    Dado que existe uma área marítima cadastrada
    Quando envio um GET para "/operacoes-ms/areas-maritimas"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "area-maritima-lista.json"
    E a lista da resposta deve conter o item com id "{areaId}"
```

```
Cenário: Atualizar o nível de poluição de uma área
  Dado que existe uma área marítima cadastrada
  Quando envio um PUT para "/operacoes-ms/areas-maritimas/{areaId}" com o corpo:
    """
    {"nome": "Área limpa {unico}", "localizacao": "Santos - SP", "nivelPoluicao": "BAIXO"}
    """
  Então o status da resposta deve ser 200
  E a resposta deve respeitar o contrato "area-maritima.json"
  E a resposta deve conter:
    | id            | {areaId}           |
    | nome          | Área limpa {unico} |
    | nivelPoluicao | BAIXO              |
```

```
Cenário: Remover uma área marítima sem operações
  Dado que existe uma área marítima cadastrada
  Quando envio um DELETE para "/operacoes-ms/areas-maritimas/{areaId}"
  Então o status da resposta deve ser 204
  Quando envio um GET para "/operacoes-ms/areas-maritimas/{areaId}"
  Então o status da resposta deve ser 404
  E a resposta deve respeitar o contrato "erro.json"
  E a resposta deve conter:
    | erro | Área marítima não encontrada com id: {areaId} |
```

```
Esquema do Cenário: Rejeitar cadastro sem o campo obrigatório "<campo>"
  Quando envio um POST para "/operacoes-ms/areas-maritimas" com o corpo:
    """
    <corpo>
    """
  Então o status da resposta deve ser 400
  E a resposta deve respeitar o contrato "erro-validacao.json"
  E a resposta deve conter:
    | <campo> | <mensagem> |

  Exemplos:
    | campo         | corpo                                                          | mensagem                          |
    | nome          | {"localizacao": "Santos - SP", "nivelPoluicao": "ALTO"}        | Nome é obrigatório                |
    | localizacao   | {"nome": "Praia X", "nivelPoluicao": "ALTO"}                   | A localização é obrigatória       |
    | nivelPoluicao | {"nome": "Praia X", "localizacao": "Santos - SP"}              | Nível de poluição é obrigatório   |
```

```
Cenário: Consultar uma área marítima inexistente
  Quando envio um GET para "/operacoes-ms/areas-maritimas/999999999"
  Então o status da resposta deve ser 404
  E a resposta deve respeitar o contrato "erro.json"
  E a resposta deve conter:
    | erro | Área marítima não encontrada com id: 999999999 |
```

```
Cenário: Cadastrar área sem autenticação
  Dado que não estou autenticado
  Quando envio um POST para "/operacoes-ms/areas-maritimas" com o corpo:
    """
    {"nome": "Praia sem login", "localizacao": "Santos - SP", "nivelPoluicao": "ALTO"}
    """
  Então o status da resposta deve ser 401
```

```
Cenário: Impedir a remoção de uma área com operações vinculadas
  Dado que existe uma área marítima cadastrada
  E que existe uma operação "PLANEJADA" cadastrada nessa área
  Quando envio um DELETE para "/operacoes-ms/areas-maritimas/{areaId}"
  Então o status da resposta deve ser 409
  E a resposta deve respeitar o contrato "erro.json"
  E o campo "erro" da resposta deve conter o texto "existem 1 operação(ões) vinculada(s)"
```

Destaques de comportamento

- CRUD completo e consulta por id.
- 400 para nome, localização e nível de poluição ausentes.
- 401 sem credenciais; 404 para id inexistente; 409 para área com operação.

3.3 Gestão de voluntários

Cobre CRUD, unicidade de e-mail, três casos de dados inválidos, autenticação, recurso inexistente e bloqueio de exclusão com relatório.

Cenários expandidos e resultado

#	Cenário	Tags	Resultado
1	Cadastrar um voluntário válido e consultá-lo pelo id	@voluntarios-ms @voluntarios @principal	PASSOU
2	Listar os voluntários	@voluntarios-ms @voluntarios	PASSOU
3	Atualizar os dados de um voluntário	@voluntarios-ms @voluntarios	PASSOU
4	Remover um voluntário sem relatórios	@voluntarios-ms @voluntarios	PASSOU
5	Rejeitar cadastro com e-mail já utilizado	@voluntarios-ms @voluntarios @principal	PASSOU
6	Rejeitar cadastro com dados inválidos (sem nome)	@voluntarios-ms @voluntarios	PASSOU
7	Rejeitar cadastro com dados inválidos (sem e-mail)	@voluntarios-ms @voluntarios	PASSOU
8	Rejeitar cadastro com dados inválidos (e-mail inválido)	@voluntarios-ms @voluntarios	PASSOU
9	Consultar um voluntário inexistente	@voluntarios-ms @voluntarios	PASSOU
10	Cadastrar voluntário sem autenticação	@voluntarios-ms @voluntarios	PASSOU
11	Impedir a remoção de um voluntário com relatórios	@voluntarios-ms @voluntarios	PASSOU

Gherkin completo

```
# language: pt
@voluntarios-ms @voluntarios
Funcionalidade: Gestão de voluntários
  Como coordenador de limpeza
  Quero cadastrar e manter os voluntários
  Para registrar quem participou de cada operação

  Contexto:
    Dado que estou autenticado como administrador
```

```
  @principal
  Cenário: Cadastrar um voluntário válido e consultá-lo pelo id
    Quando envio um POST para "/voluntarios-ms/voluntarios" com o corpo:
      """
      {"nome": "Ana Souza", "email": "ana.{unico}@email.com", "telefone": "13988887777", "especialidade": "Mergulho"}
      """
    Então o status da resposta deve ser 201
    E a resposta deve respeitar o contrato "voluntario.json"
    E a resposta deve conter:
      | nome          | Ana Souza             |
      | email         | ana.{unico}@email.com |
      | especialidade | Mergulho              |
    E guardo o campo "id" da resposta como "voluntarioId"
    Quando envio um GET para "/voluntarios-ms/voluntarios/{voluntarioId}"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "voluntario.json"
    E a resposta deve conter:
      | id    | {voluntarioId}        |
      | email | ana.{unico}@email.com |
```

```
  Cenário: Listar os voluntários
    Dado que existe um voluntário cadastrado
    Quando envio um GET para "/voluntarios-ms/voluntarios"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "voluntario-lista.json"
    E a lista da resposta deve conter o item com id "{voluntarioId}"
```

```
Cenário: Atualizar os dados de um voluntário
  Dado que existe um voluntário cadastrado
  Quando envio um PUT para "/voluntarios-ms/voluntarios/{voluntarioId}" com o corpo:
    """
    {"nome": "Nome Atualizado", "email": "atualizado.{unico}@email.com", "especialidade": "Barco"}
    """
  Então o status da resposta deve ser 200
  E a resposta deve respeitar o contrato "voluntario.json"
  E a resposta deve conter:
    | id           | {voluntarioId}              |
    | nome         | Nome Atualizado             |
    | email        | atualizado.{unico}@email.com |
    | especialidade | Barco                      |
```

```
Cenário: Remover um voluntário sem relatórios
  Dado que existe um voluntário cadastrado
  Quando envio um DELETE para "/voluntarios-ms/voluntarios/{voluntarioId}"
  Então o status da resposta deve ser 204
  Quando envio um GET para "/voluntarios-ms/voluntarios/{voluntarioId}"
  Então o status da resposta deve ser 404
  E a resposta deve respeitar o contrato "erro.json"
```

```
@principal
Cenário: Rejeitar cadastro com e-mail já utilizado
  Dado que existe um voluntário cadastrado
  Quando envio um POST para "/voluntarios-ms/voluntarios" com o corpo:
    """
    {"nome": "Outra Pessoa", "email": "voluntario.{unico}@teste.com"}
    """
  Então o status da resposta deve ser 409
  E a resposta deve respeitar o contrato "erro.json"
  E a resposta deve conter:
    | erro | E-mail já cadastrado: voluntario.{unico}@teste.com |
```

```
Esquema do Cenário: Rejeitar cadastro com dados inválidos (<caso>)
  Quando envio um POST para "/voluntarios-ms/voluntarios" com o corpo:
    """
    <corpo>
    """
  Então o status da resposta deve ser 400
  E a resposta deve respeitar o contrato "erro-validacao.json"
  E a resposta deve conter:
    | <campo> | <mensagem> |

  Exemplos:
    | caso            | corpo                                          | campo | mensagem            |
    | sem nome        | {"email": "sem.nome@email.com"}                | nome  | Nome é obrigatório  |
    | sem e-mail      | {"nome": "Sem Email"}                          | email | Email é obrigatório |
    | e-mail inválido | {"nome": "Email Ruim", "email": "nao-e-email"} | email | Email inválido      |
```

```
Cenário: Consultar um voluntário inexistente
  Quando envio um GET para "/voluntarios-ms/voluntarios/999999999"
  Então o status da resposta deve ser 404
  E a resposta deve respeitar o contrato "erro.json"
  E a resposta deve conter:
    | erro | Voluntário não encontrado com id: 999999999 |
```

```
Cenário: Cadastrar voluntário sem autenticação
  Dado que não estou autenticado
  Quando envio um POST para "/voluntarios-ms/voluntarios" com o corpo:
    """
    {"nome": "Sem Login", "email": "sem.login.{unico}@email.com"}
    """
  Então o status da resposta deve ser 401
```

```
Cenário: Impedir a remoção de um voluntário com relatórios
  Dado que existe uma área marítima cadastrada
  E que existe um voluntário cadastrado
  E que existe uma operação "EM_ANDAMENTO" cadastrada nessa área
  E que existe um relatório de coleta dessa operação
  Quando envio um DELETE para "/voluntarios-ms/voluntarios/{voluntarioId}"
  Então o status da resposta deve ser 409
  E a resposta deve respeitar o contrato "erro.json"
  E o campo "erro" da resposta deve conter o texto "existem 1 relatório(s) vinculado(s)"
```

Destaques de comportamento

- CRUD completo, unicidade de e-mail e três casos de validação.
- 401 sem credenciais; 404 para id inexistente; 409 para voluntário com relatório.

3.4 Operações de limpeza e integração com voluntários

Cobre CRUD, filtro por status, conclusão com criação de relatório no voluntarios-ms, fail-fast, falha 502, bloqueio de exclusão e validações.

Cenários expandidos e resultado

#	Cenário	Tags	Resultado
1	Planejar uma operação em uma área existente	@operacoes-ms @operacoes	PASSOU
2	Listar todas as operações	@operacoes-ms @operacoes	PASSOU
3	Filtrar operações por status	@operacoes-ms @operacoes	PASSOU
4	Atualizar uma operação sem concluí-la	@operacoes-ms @operacoes	PASSOU
5	Remover uma operação sem relatórios	@operacoes-ms @operacoes	PASSOU
6	Concluir uma operação gera o relatório de coleta no voluntarios-ms	@operacoes-ms @operacoes @principal @integracao	PASSOU
7	Rejeitar conclusão sem os dados da coleta	@operacoes-ms @operacoes @principal @integracao	PASSOU
8	Concluir com voluntário inexistente retorna falha de comunicação com o voluntarios-ms	@operacoes-ms @operacoes @integracao	PASSOU
9	Impedir a remoção de uma operação que já tem relatório	@operacoes-ms @operacoes @integracao	PASSOU
10	Rejeitar operação em área inexistente	@operacoes-ms @operacoes	PASSOU
11	Rejeitar operação sem os campos obrigatórios	@operacoes-ms @operacoes	PASSOU
12	Consultar uma operação inexistente	@operacoes-ms @operacoes	PASSOU
13	Excluir operação sem autenticação	@operacoes-ms @operacoes	PASSOU

Gherkin completo

```
# language: pt
@operacoes-ms @operacoes
Funcionalidade: Operações de limpeza e integração com voluntários
  Como coordenador de limpeza
  Quero planejar, acompanhar e concluir operações de limpeza
  Para que o resíduo coletado fique registrado no relatório do voluntário

  Contexto:
    Dado que estou autenticado como administrador
    E que existe uma área marítima cadastrada
```

```
  Cenário: Planejar uma operação em uma área existente
    Quando envio um POST para "/operacoes-ms/operacoes" com o corpo:
      """
      {"titulo": "Mutirão {unico}", "descricao": "Limpeza da faixa de areia",
       "dataOperacao": "2026-12-10", "status": "PLANEJADA", "idArea": {areaId}}
      """
    Então o status da resposta deve ser 201
    E a resposta deve respeitar o contrato "operacao.json"
    E a resposta deve conter:
      | titulo       | Mutirão {unico} |
      | status       | PLANEJADA       |
      | dataOperacao | 2026-12-10      |
      | idArea       | {areaId}        |
      | nomeArea     | Área de teste {unico} |
    E guardo o campo "id" da resposta como "operacaoId"
    Quando envio um GET para "/operacoes-ms/operacoes/{operacaoId}"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "operacao.json"
    E a resposta deve conter:
      | id | {operacaoId} |
```

```
  Cenário: Listar todas as operações
    Dado que existe uma operação "PLANEJADA" cadastrada nessa área
    Quando envio um GET para "/operacoes-ms/operacoes"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "operacao-lista.json"
    E a lista da resposta deve conter o item com id "{operacaoId}"
```

```
  Cenário: Filtrar operações por status
    Dado que existe uma operação "EM_ANDAMENTO" cadastrada nessa área
    Quando envio um GET para "/operacoes-ms/operacoes/status/EM_ANDAMENTO"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "operacao-lista.json"
    E a lista da resposta deve conter o item com id "{operacaoId}"
    E todos os itens da lista devem ter "status" igual a "EM_ANDAMENTO"
    Quando envio um GET para "/operacoes-ms/operacoes/status/CANCELADA"
    Então o status da resposta deve ser 200
    E a lista da resposta não deve conter o item com id "{operacaoId}"
```

```
  Cenário: Atualizar uma operação sem concluí-la
    Dado que existe uma operação "PLANEJADA" cadastrada nessa área
    Quando envio um PUT para "/operacoes-ms/operacoes/{operacaoId}" com o corpo:
      """
      {"titulo": "Mutirão remarcado", "dataOperacao": "2026-12-20", "status": "EM_ANDAMENTO", "idArea": {areaId}}
      """
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "operacao.json"
    E a resposta deve conter:
      | titulo       | Mutirão remarcado |
      | status       | EM_ANDAMENTO      |
      | dataOperacao | 2026-12-20        |
    Quando envio um GET para "/voluntarios-ms/relatorios/operacao/{operacaoId}"
    Então o status da resposta deve ser 200
    E a lista da resposta deve ter 0 itens
```

```
  Cenário: Remover uma operação sem relatórios
    Dado que existe uma operação "PLANEJADA" cadastrada nessa área
    Quando envio um DELETE para "/operacoes-ms/operacoes/{operacaoId}"
    Então o status da resposta deve ser 204
    Quando envio um GET para "/operacoes-ms/operacoes/{operacaoId}"
    Então o status da resposta deve ser 404
    E a resposta deve respeitar o contrato "erro.json"
    E a resposta deve conter:
      | erro | Operação não encontrada com id: {operacaoId} |

  # ------------------------------------------------------------------ comunicação entre serviços
```

```
@principal @integracao
Cenário: Concluir uma operação gera o relatório de coleta no voluntarios-ms
  Dado que existe um voluntário cadastrado
  E que existe uma operação "EM_ANDAMENTO" cadastrada nessa área
  Quando envio um PUT para "/operacoes-ms/operacoes/{operacaoId}" com o corpo:
    """
    {"titulo": "Mutirão {unico}", "dataOperacao": "2026-12-10", "status": "CONCLUIDA", "idArea": {areaId},
     "idVoluntario": {voluntarioId}, "quantidadeResiduos": 120, "tipoResiduo": "Plástico",
     "observacoes": "Concluída pelos testes"}
    """
  Então o status da resposta deve ser 200
  E a resposta deve respeitar o contrato "operacao.json"
  E a resposta deve conter:
    | status | CONCLUIDA |
  Quando envio um GET para "/voluntarios-ms/relatorios/operacao/{operacaoId}"
  Então o status da resposta deve ser 200
  E a resposta deve respeitar o contrato "relatorio-lista.json"
  E a lista da resposta deve ter 1 item
  E a resposta deve conter:
    | [0].idOperacao         | {operacaoId}            |
    | [0].idVoluntario       | {voluntarioId}          |
    | [0].nomeVoluntario     | Voluntário {unico}      |
    | [0].quantidadeResiduos | 120                     |
    | [0].tipoResiduo        | Plástico                |
    | [0].observacoes        | Concluída pelos testes  |
```

```
@principal @integracao
Cenário: Rejeitar conclusão sem os dados da coleta
  Dado que existe uma operação "EM_ANDAMENTO" cadastrada nessa área
  Quando envio um PUT para "/operacoes-ms/operacoes/{operacaoId}" com o corpo:
    """
    {"titulo": "Mutirão {unico}", "dataOperacao": "2026-12-10", "status": "CONCLUIDA", "idArea": {areaId}}
    """
  Então o status da resposta deve ser 409
  E a resposta deve respeitar o contrato "erro.json"
  E a resposta deve conter:
    | erro | Conclusão exige idVoluntario, quantidadeResiduos e tipoResiduo |
  Quando envio um GET para "/voluntarios-ms/relatorios/operacao/{operacaoId}"
  Então a lista da resposta deve ter 0 itens
```

```
@integracao
Cenário: Concluir com voluntário inexistente retorna falha de comunicação com o voluntarios-ms
  Dado que existe uma operação "EM_ANDAMENTO" cadastrada nessa área
  Quando envio um PUT para "/operacoes-ms/operacoes/{operacaoId}" com o corpo:
    """
    {"titulo": "Mutirão {unico}", "dataOperacao": "2026-12-10", "status": "CONCLUIDA", "idArea": {areaId},
     "idVoluntario": 999999999, "quantidadeResiduos": 10, "tipoResiduo": "Metal"}
    """
  Então o status da resposta deve ser 502
  E a resposta deve respeitar o contrato "erro.json"
  E o campo "erro" da resposta deve conter o texto "Falha voluntarios-ms"
```

```
@integracao
Cenário: Impedir a remoção de uma operação que já tem relatório
  Dado que existe um voluntário cadastrado
  E que existe uma operação "EM_ANDAMENTO" cadastrada nessa área
  E que essa operação foi concluída pelo voluntário com 80 kg de "Vidro"
  Quando envio um DELETE para "/operacoes-ms/operacoes/{operacaoId}"
  Então o status da resposta deve ser 409
  E a resposta deve respeitar o contrato "erro.json"
  E a resposta deve conter:
    | erro | Não é possível excluir a operação pois existem relatórios vinculados a ela |

# ---------------------------------------------------------------- validações
```

```
Cenário: Rejeitar operação em área inexistente
  Quando envio um POST para "/operacoes-ms/operacoes" com o corpo:
    """
    {"titulo": "Área fantasma", "dataOperacao": "2026-12-10", "status": "PLANEJADA", "idArea": 999999999}
    """
  Então o status da resposta deve ser 404
  E a resposta deve respeitar o contrato "erro.json"
  E a resposta deve conter:
    | erro | Área marítima não encontrada com id: 999999999 |
```

```
Cenário: Rejeitar operação sem os campos obrigatórios
  Quando envio um POST para "/operacoes-ms/operacoes" com o corpo:
    """
    {"descricao": "sem nada"}
    """
  Então o status da resposta deve ser 400
  E a resposta deve respeitar o contrato "erro-validacao.json"
  E a resposta deve conter:
    | titulo       | Título é obrigatório             |
    | dataOperacao | Data da operação é obrigatória   |
    | status       | Status é obrigatório             |
    | idArea       | ID da área é obrigatório         |
```

```
            Cenário: Consultar uma operação inexistente
              Quando envio um GET para "/operacoes-ms/operacoes/999999999"
              Então o status da resposta deve ser 404
              E a resposta deve respeitar o contrato "erro.json"
```

```
            Cenário: Excluir operação sem autenticação
              Dado que existe uma operação "PLANEJADA" cadastrada nessa área
              E que não estou autenticado
              Quando envio um DELETE para "/operacoes-ms/operacoes/{operacaoId}"
              Então o status da resposta deve ser 401
```

Destaques de comportamento

- Conclusão bem-sucedida cria relatório no voluntarios-ms via Feign.
- Conclusão sem dados de coleta retorna 409 e não cria relatório.
- Voluntário inexistente na conclusão retorna 502 (falha de comunicação).
- Exclusão de operação com relatório é bloqueada com 409.

3.5 Relatórios de coleta

Cobre registro manual, listagem, filtros por operação/voluntário, exclusão, validações, 404 e autenticação.

Cenários expandidos e resultado

#	Cenário	Tags	Resultado
1	Registrar um relatório de coleta manualmente	@voluntarios-ms @relatorios	PASSOU
2	Listar todos os relatórios	@voluntarios-ms @relatorios	PASSOU
3	Consultar relatórios por operação e por voluntário	@voluntarios-ms @relatorios	PASSOU
4	Remover um relatório	@voluntarios-ms @relatorios	PASSOU
5	Rejeitar relatório de voluntário inexistente	@voluntarios-ms @relatorios	PASSOU
6	Rejeitar relatório sem os campos obrigatórios	@voluntarios-ms @relatorios	PASSOU
7	Consultar relatórios de uma operação sem coleta	@voluntarios-ms @relatorios	PASSOU
8	Remover relatório sem autenticação	@voluntarios-ms @relatorios	PASSOU

Gherkin completo

```
# language: pt
@voluntarios-ms @relatorios
Funcionalidade: Relatórios de coleta
  Como coordenador de limpeza
  Quero registrar e consultar os relatórios de coleta
  Para acompanhar quanto resíduo cada operação e cada voluntário removeram

  Contexto:
    Dado que estou autenticado como administrador
    E que existe uma área marítima cadastrada
    E que existe um voluntário cadastrado
    E que existe uma operação "EM_ANDAMENTO" cadastrada nessa área
```

```
  Cenário: Registrar um relatório de coleta manualmente
    Quando envio um POST para "/voluntarios-ms/relatorios" com o corpo:
      """
      {"idOperacao": {operacaoId}, "idVoluntario": {voluntarioId},
       "quantidadeResiduos": 42, "tipoResiduo": "Redes de pesca", "observacoes": "Coleta manual"}
      """
    Então o status da resposta deve ser 201
    E a resposta deve respeitar o contrato "relatorio.json"
    E a resposta deve conter:
      | idOperacao         | {operacaoId}       |
      | idVoluntario       | {voluntarioId}     |
      | nomeVoluntario     | Voluntário {unico} |
      | quantidadeResiduos | 42                 |
      | tipoResiduo        | Redes de pesca     |
    E guardo o campo "id" da resposta como "relatorioId"
    Quando envio um GET para "/voluntarios-ms/relatorios/{relatorioId}"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "relatorio.json"
    E a resposta deve conter:
      | id | {relatorioId} |
```

```
  Cenário: Listar todos os relatórios
    Dado que existe um relatório de coleta dessa operação
    Quando envio um GET para "/voluntarios-ms/relatorios"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "relatorio-lista.json"
    E a lista da resposta deve conter o item com id "{relatorioId}"
```

```
Cenário: Consultar relatórios por operação e por voluntário
  Dado que existe um relatório de coleta dessa operação
  Quando envio um GET para "/voluntarios-ms/relatorios/operacao/{operacaoId}"
  Então o status da resposta deve ser 200
  E a resposta deve respeitar o contrato "relatorio-lista.json"
  E a lista da resposta deve ter 1 item
  E a lista da resposta deve conter o item com id "{relatorioId}"
  Quando envio um GET para "/voluntarios-ms/relatorios/voluntario/{voluntarioId}"
  Então o status da resposta deve ser 200
  E a resposta deve respeitar o contrato "relatorio-lista.json"
  E a lista da resposta deve ter 1 item
  E todos os itens da lista devem ter "idVoluntario" igual a "{voluntarioId}"
```

```
Cenário: Remover um relatório
  Dado que existe um relatório de coleta dessa operação
  Quando envio um DELETE para "/voluntarios-ms/relatorios/{relatorioId}"
  Então o status da resposta deve ser 204
  Quando envio um GET para "/voluntarios-ms/relatorios/{relatorioId}"
  Então o status da resposta deve ser 404
  E a resposta deve respeitar o contrato "erro.json"
  E a resposta deve conter:
    | erro | Relatório não encontrado com id: {relatorioId} |
```

```
Cenário: Rejeitar relatório de voluntário inexistente
  Quando envio um POST para "/voluntarios-ms/relatorios" com o corpo:
    """
    {"idOperacao": {operacaoId}, "idVoluntario": 999999999, "quantidadeResiduos": 5, "tipoResiduo": "Metal"}
    """
  Então o status da resposta deve ser 404
  E a resposta deve respeitar o contrato "erro.json"
  E a resposta deve conter:
    | erro | Voluntário não encontrado com id: 999999999 |
```

```
Cenário: Rejeitar relatório sem os campos obrigatórios
  Quando envio um POST para "/voluntarios-ms/relatorios" com o corpo:
    """
    {"observacoes": "incompleto"}
    """
  Então o status da resposta deve ser 400
  E a resposta deve respeitar o contrato "erro-validacao.json"
  E a resposta deve conter:
    | idOperacao         | ID da operação é obrigatório        |
    | idVoluntario       | ID do voluntário é obrigatório      |
    | quantidadeResiduos | Quantidade de resíduos é obrigatória |
    | tipoResiduo        | Tipo de resíduo é obrigatório       |
```

```
Cenário: Consultar relatórios de uma operação sem coleta
  Quando envio um GET para "/voluntarios-ms/relatorios/operacao/{operacaoId}"
  Então o status da resposta deve ser 200
  E a resposta deve respeitar o contrato "relatorio-lista.json"
  E a lista da resposta deve ter 0 itens
```

```
Cenário: Remover relatório sem autenticação
  Dado que existe um relatório de coleta dessa operação
  E que não estou autenticado
  Quando envio um DELETE para "/voluntarios-ms/relatorios/{relatorioId}"
  Então o status da resposta deve ser 401
```

Destaques de comportamento

- Registro manual de relatório e consultas por operação/voluntário.
- Validações 400, 404 e 401, além de exclusão e lista vazia.

4. Cobertura das APIs e contratos

A suíte cobre os endpoints de negócio e também a infraestrutura de entrada (Gateway/Eureka). A tabela abaixo relaciona cada endpoint à feature que o exercita.

4.1 Matriz de endpoints

Serviço	Método	Rota	Feature
gateway	GET	`/actuator/health`	01
gateway	GET	`/actuator/info`	01
eureka-sd	GET	`/eureka/apps`	01
operacoes-ms	POST	`/areas-maritimas`	02
operacoes-ms	GET	`/areas-maritimas`	02
operacoes-ms	GET	`/areas-maritimas/{id}`	02
operacoes-ms	PUT	`/areas-maritimas/{id}`	02
operacoes-ms	DELETE	`/areas-maritimas/{id}`	02
operacoes-ms	POST	`/operacoes`	04
operacoes-ms	GET	`/operacoes`	04
operacoes-ms	GET	`/operacoes/{id}`	04
operacoes-ms	GET	`/operacoes/status/{status}`	04
operacoes-ms	PUT	`/operacoes/{id}`	04
operacoes-ms	DELETE	`/operacoes/{id}`	04
voluntarios-ms	POST	`/voluntarios`	03
voluntarios-ms	GET	`/voluntarios`	03
voluntarios-ms	GET	`/voluntarios/{id}`	03
voluntarios-ms	PUT	`/voluntarios/{id}`	03
voluntarios-ms	DELETE	`/voluntarios/{id}`	03
voluntarios-ms	POST	`/relatorios`	05
voluntarios-ms	GET	`/relatorios`	05
voluntarios-ms	GET	`/relatorios/{id}`	05
voluntarios-ms	GET	`/relatorios/operacao/{id}`	05
voluntarios-ms	GET	`/relatorios/voluntario/{id}`	05
voluntarios-ms	DELETE	`/relatorios/{id}`	05

4.2 Contratos JSON Schema

#	Schema	Uso
1	`area-maritima.json`	Área
2	`area-maritima-lista.json`	Lista de áreas
3	`operacao.json`	Operação
4	`operacao-lista.json`	Lista de operações
5	`voluntario.json`	Voluntário
6	`voluntario-lista.json`	Lista de voluntários
7	`relatorio.json`	Relatório
8	`relatorio-lista.json`	Lista de relatórios
9	`erro.json`	Erro de negócio
10	`erro-validacao.json`	Erro de validação
11	`health.json`	Actuator health
12	`info.json`	Actuator info
13	`eureka-apps.json`	Registro do Eureka

4.3 Comunicação entre serviços

O cenário principal conclui uma operação no operacoes-ms e consulta o relatório no voluntarios-ms pelo Gateway. Ele valida o efeito real do Feign, não um mock: o relatório criado contém ids, voluntário, quantidade e tipo de resíduo. Outros cenários cobrem o fail-fast 409, a falha de comunicação 502 e o bloqueio de exclusão 409.

5. Configurações da suíte e do CI

Os prints abaixo são trechos reais dos arquivos versionados. A suíte é um projeto Maven independente em api-tests e não depende do código dos microsserviços.

5.1 Dependências da suíte

Cucumber 7, Rest Assured, JUnit 5 e o validador de JSON Schema. O Surefire executa o runner RunCucumberTest.

api-tests/pom.xml — Cucumber + Rest Assured

```
        <dependencies>
            <dependency>
                <groupId>io.cucumber</groupId>
                <artifactId>cucumber-bom</artifactId>
                <version>${cucumber.version}</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
            <dependency>
                <groupId>org.junit</groupId>
                <artifactId>junit-bom</artifactId>
                <version>${junit.version}</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
```

Figura 2 — Print do api-tests/pom.xml.

5.2 Script de execução

O script lê as credenciais do .env, aceita --subir/--derrubar e repassa filtros do Cucumber ao Maven.

api-tests/executar.sh — execução local/CI

```
#!/usr/bin/env bash
# Executa os testes de API (BDD + contrato) contra o ambiente do Ocean Cleaner.
#
# Uso:
#   ./api-tests/executar.sh             # testa o ambiente já no ar (padrão: http://localhost:5051)
#   ./api-tests/executar.sh --subir     # sobe o docker compose local antes de testar
#   ./api-tests/executar.sh --subir --derrubar   # ... e derruba tudo (inclusive o banco) no fim
#
# Variáveis opcionais: BASE_URL, EUREKA_URL, API_USER, API_PASSWORD, READY_TIMEOUT_SECONDS.
# Se API_USER/API_PASSWORD não forem informados, são lidos do .env da raiz do projeto.
# Argumentos extras vão para o Maven, ex.: ./api-tests/executar.sh -Dcucumber.filter.tags=@principal
set -euo pipefail

RAIZ="$(cd "$(dirname "$0")/.." && pwd)"
SUBIR=false
DERRUBAR=false
MAVEN_ARGS=()
for arg in "$@"; do
  case "$arg" in
    --subir) SUBIR=true ;;
    --derrubar) DERRUBAR=true ;;
    *) MAVEN_ARGS+=("$arg") ;;
  esac
done

if [[ -f "$RAIZ/.env" ]]; then
  : "${API_USER:=$(grep -E '^API_USER=' "$RAIZ/.env" | cut -d= -f2-)}"
  : "${API_PASSWORD:=$(grep -E '^API_PASSWORD=' "$RAIZ/.env" | cut -d= -f2-)}"
fi
export API_USER API_PASSWORD

if [[ -z "${API_PASSWORD:-}" ]]; then
  echo "API_PASSWORD não definido (crie o .env a partir do .env.example ou exporte a variável)." >&2
  exit 1
fi

if $DERRUBAR; then
  trap 'docker compose -f "$RAIZ/docker-compose.yaml" --project-directory "$RAIZ" down -v' EXIT
fi

if $SUBIR; then
  docker compose -f "$RAIZ/docker-compose.yaml" --project-directory "$RAIZ" up -d --build
fi

cd "$RAIZ/api-tests"
sh ./mvnw -B test "${MAVEN_ARGS[@]}"
echo "Relatório: $RAIZ/api-tests/target/cucumber-report.html"
```

Figura 3 — Print do api-tests/executar.sh.

5.3 Feature com o cenário de integração

O trecho abaixo mostra o cenário que conclui a operação e verifica o relatório no voluntarios-ms via Gateway.

04_operacoes.feature — cenário de integração Feign

```
@principal @integracao
Cenário: Concluir uma operação gera o relatório de coleta no voluntarios-ms
  Dado que existe um voluntário cadastrado
  E que existe uma operação "EM_ANDAMENTO" cadastrada nessa área
  Quando envio um PUT para "/operacoes-ms/operacoes/{operacaoId}" com o corpo:
    """
    {"titulo": "Mutirão {unico}", "dataOperacao": "2026-12-10", "status": "CONCLUIDA", "idArea": {areaId},
     "idVoluntario": {voluntarioId}, "quantidadeResiduos": 120, "tipoResiduo": "Plástico",
     "observacoes": "Concluída pelos testes"}
    """
  Então o status da resposta deve ser 200
  E a resposta deve respeitar o contrato "operacao.json"
  E a resposta deve conter:
    | status | CONCLUIDA |
  Quando envio um GET para "/voluntarios-ms/relatorios/operacao/{operacaoId}"
  Então o status da resposta deve ser 200
  E a resposta deve respeitar o contrato "relatorio-lista.json"
  E a lista da resposta deve ter 1 item
  E a resposta deve conter:
    | [0].idOperacao         | {operacaoId}           |
    | [0].idVoluntario       | {voluntarioId}         |
    | [0].nomeVoluntario     | Voluntário {unico}     |
    | [0].quantidadeResiduos | 120                    |
    | [0].tipoResiduo        | Plástico               |
    | [0].observacoes        | Concluída pelos testes |

@principal @integracao
Cenário: Rejeitar conclusão sem os dados da coleta
  Dado que existe uma operação "EM_ANDAMENTO" cadastrada nessa área
  Quando envio um PUT para "/operacoes-ms/operacoes/{operacaoId}" com o corpo:
    """
    {"titulo": "Mutirão {unico}", "dataOperacao": "2026-12-10", "status": "CONCLUIDA", "idArea": {areaId}}
    """
  Então o status da resposta deve ser 409
  E a resposta deve respeitar o contrato "erro.json"
  E a resposta deve conter:
    | erro | Conclusão exige idVoluntario, quantidadeResiduos e tipoResiduo |
  Quando envio um GET para "/voluntarios-ms/relatorios/operacao/{operacaoId}"
  Então a lista da resposta deve ter 0 itens

@integracao
Cenário: Concluir com voluntário inexistente retorna falha de comunicação com o voluntarios-ms
  Dado que existe uma operação "EM_ANDAMENTO" cadastrada nessa área
  Quando envio um PUT para "/operacoes-ms/operacoes/{operacaoId}" com o corpo:
    """
    {"titulo": "Mutirão {unico}", "dataOperacao": "2026-12-10", "status": "CONCLUIDA", "idArea": {areaId},
     "idVoluntario": 999999999, "quantidadeResiduos": 10, "tipoResiduo": "Metal"}
    """
  Então o status da resposta deve ser 502
  E a resposta deve respeitar o contrato "erro.json"
  E o campo "erro" da resposta deve conter o texto "Falha voluntarios-ms"

```

Figura 4 — Print de 04_operacoes.feature.

5.4 Contrato JSON Schema

Exemplo do contrato de operação, com campos obrigatórios, tipos, formatos e additionalProperties: false.

api-tests/schemas/operacao.json — contrato JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "title": "Operação de limpeza (OperacaoExibicaoDto)",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "id",
    "titulo",
    "descricao",
    "dataOperacao",
    "status",
    "idArea",
    "nomeArea"
  ],
  "properties": {
    "id": {
      "type": "integer",
      "minimum": 1
    },
    "titulo": {
      "type": "string",
      "minLength": 1
    },
    "descricao": {
      "type": [
        "string",
        "null"
      ]
    },
    "dataOperacao": {
      "type": "string",
      "pattern": "^\\d{4}-\\d{2}-\\d{2}$"
    },
    "status": {
      "type": "string",
      "minLength": 1
    },
    "idArea": {
      "type": "integer",
      "minimum": 1
    },
    "nomeArea": {
      "type": "string",
      "minLength": 1
    }
  }
}
```

Figura 5 — Print de schemas/operacao.json.

5.5 Job de CI

O job api-tests sobe o Docker Compose com senhas aleatórias, executa a suíte, publica o artifact e derruba o ambiente com volume.

.github/workflows/ci-cd.yml — job api-tests

```yaml
  # 2) TESTES DE API: sobe o ambiente completo (Oracle + Eureka + Gateway + MS)
  #    com docker compose e roda os cenários BDD (Cucumber + Rest Assured):
  #    status code, corpo JSON, contrato (JSON Schema) e comunicação entre serviços.
  # ------------------------------------------------------------------------
  api-tests:
    name: Testes de API (BDD + contrato)
    needs: build-test
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v4

      - name: Configurar Java 21
        uses: actions/setup-java@v5
        with:
          distribution: temurin
          java-version: '21'
          cache: maven
          cache-dependency-path: api-tests/pom.xml

      - name: Criar .env efêmero (senhas aleatórias, só para este job)
        run: |
          cp .env.example .env
          for VAR in ORACLE_PASSWORD DB_PASSWORD API_PASSWORD; do
            SENHA="Ci$(openssl rand -hex 12)"
            echo "::add-mask::$SENHA"
            sed -i "s/^$VAR=.*/$VAR=$SENHA/" .env
          done

      - name: Subir ambiente e executar testes de API
        run: ./api-tests/executar.sh --subir

      - name: Resumo dos testes de API
        if: always()
        run: |
          echo "### Testes de API (Cucumber)" >> "$GITHUB_STEP_SUMMARY"
```

Figura 6 — Print do job api-tests no workflow.

5.6 Visão dos cenários no README da suíte

O README da suíte resume as features, os cenários positivos/negativos e a cobertura de endpoints.

api-tests/README.md — visão dos cenários

```markdown
## Cenários

45 cenários em 5 funcionalidades ([`src/test/resources/features`](src/test/resources/features)):

| Feature | Positivos | Negativos |
|---|---|---|
| `01_infraestrutura` | Gateway `health`/`info`; serviços registrados no Eureka | Rota inexistente no Gateway (404) |
| `02_areas_maritimas` | Cadastrar + buscar, listar, atualizar, remover | Campos obrigatórios (400 ×3), inexistente (404), sem login (401), área com operação (409) |
| `03_voluntarios` | Cadastrar + buscar, listar, atualizar, remover | E-mail duplicado (409), dados inválidos (400 ×3), inexistente (404), sem login (401), voluntário com relatório (409) |
| `04_operacoes` | Planejar + buscar, listar, filtrar por status, atualizar, remover, **concluir → relatório criado no voluntarios-ms** | Conclusão sem dados (409), voluntário inexistente na conclusão (*
| `05_relatorios` | Registrar + buscar, listar, por operação e por voluntário, remover, operação sem coleta (lista vazia) | Voluntário inexistente (404), obrigatórios (400), sem login (401) |

### Cobertura de endpoints

| Serviço | Endpoint | Testado em |
|---|---|---|
| gateway | `GET /actuator/health`, `GET /actuator/info` | 01 |
| eureka-sd | `GET /eureka/apps` | 01 |
| operacoes-ms | `POST/GET /areas-maritimas`, `GET/PUT/DELETE /areas-maritimas/{id}` | 02 |
| operacoes-ms | `POST/GET /operacoes`, `GET/PUT/DELETE /operacoes/{id}`, `GET /operacoes/status/{status}` | 04 |
| voluntarios-ms | `POST/GET /voluntarios`, `GET/PUT/DELETE /voluntarios/{id}` | 03 |
| voluntarios-ms | `POST/GET /relatorios`, `GET/DELETE /relatorios/{id}`, `GET /relatorios/operacao/{id}`, `GET /relatorios/voluntario/{id}` | 05 (e 04, via Feign) |

### Tags

| Tag | Seleciona |
|---|---|
| `@principal` | Os cenários centrais do negócio (felizes e de falha) |
| `@integracao` | Comunicação `operacoes-ms` → `voluntarios-ms` (Feign) |
| `@infraestrutura` | Gateway e Eureka |
| `@operacoes-ms`, `@voluntarios-ms` | Por microsserviço |
| `@areas-maritimas`, `@operacoes`, `@voluntarios`, `@relatorios` | Por recurso |

### Exemplo

```gherkin
@principal @integracao
Cenário: Concluir uma operação gera o relatório de coleta no voluntarios-ms
  Dado que existe um voluntário cadastrado
  E que existe uma operação "EM_ANDAMENTO" cadastrada nessa área
  Quando envio um PUT para "/operacoes-ms/operacoes/{operacaoId}" com o corpo:
    """
    {"titulo": "Mutirão {unico}", "dataOperacao": "2026-12-10", "status": "CONCLUIDA", "idArea": {areaId},
     "idVoluntario": {voluntarioId}, "quantidadeResiduos": 120, "tipoResiduo": "Plástico", ...}
    """
  Então o status da resposta deve ser 200
  E a resposta deve respeitar o contrato "operacao.json"
  Quando envio um GET para "/voluntarios-ms/relatorios/operacao/{operacaoId}"
```

Figura 7 — Print do README da suíte api-tests.
```

6. Evidências de execução

A execução de referência é a do GitHub Actions no commit `73ea4dd` da master. O job api-tests subiu o ambiente completo e executou a suíte em 90,93s (4m18s de job). O artifact api-tests-report foi baixado e usado nas capturas abaixo.

6.1 Resultado do CI

Item	Valor
Workflow	CI/CD - Ocean Cleaner
Run	41266518830
Job	Testes de API (BDD + contrato) — 124730480825
Commit	73ea4dd (master)
Resultado	45 testes, 0 falhas, 0 erros, 0 ignorados
Tempo	90,93s de testes; 4m18s de job
Artifact	api-tests-report (HTML, JSON, XML)

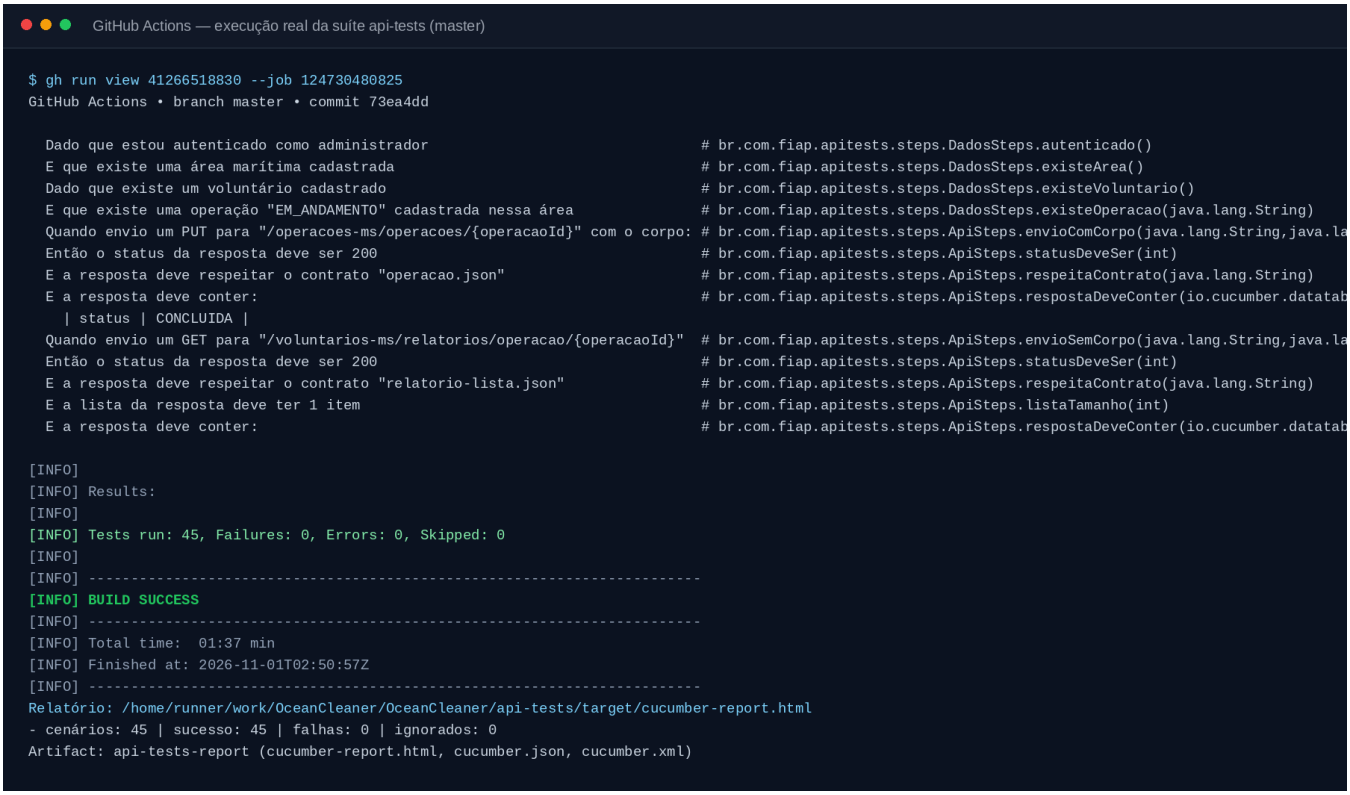


Figura 8 — Print do log real do job: 45/45, BUILD SUCCESS e artifact.

6.2 Relatório Cucumber — visão geral

O relatório HTML exibe 45 cenários executados, 100% de aprovação e a lista das cinco features.

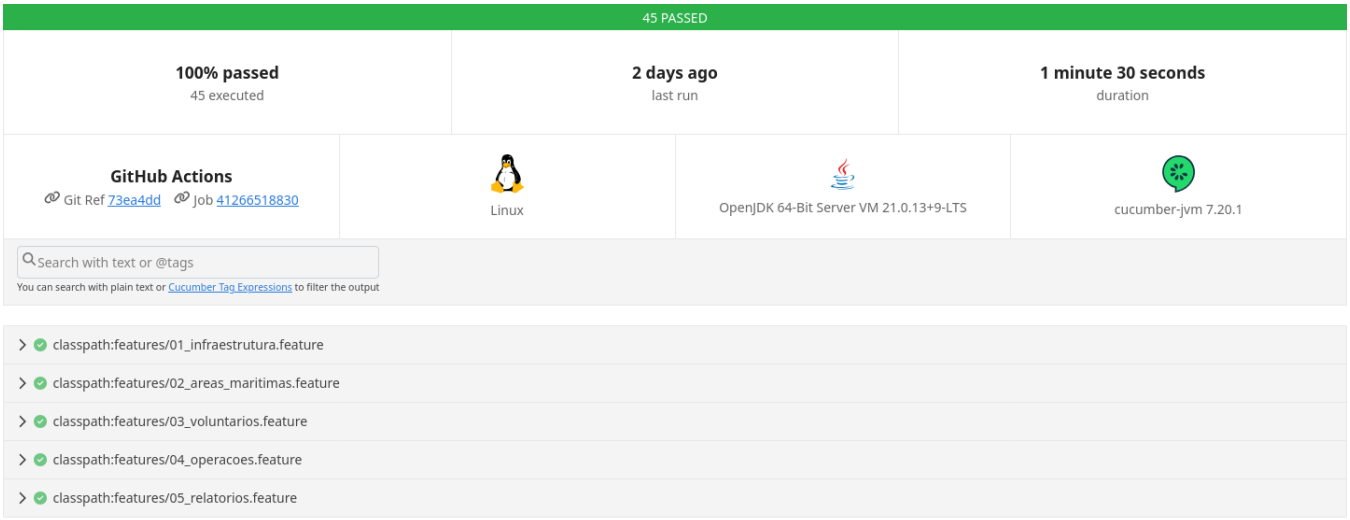


Figura 9 — Print do relatório HTML: 45 PASSED / 100%.

6.3 Relatório Cucumber — features



Figura 10 — Print da feature de operações no relatório HTML.

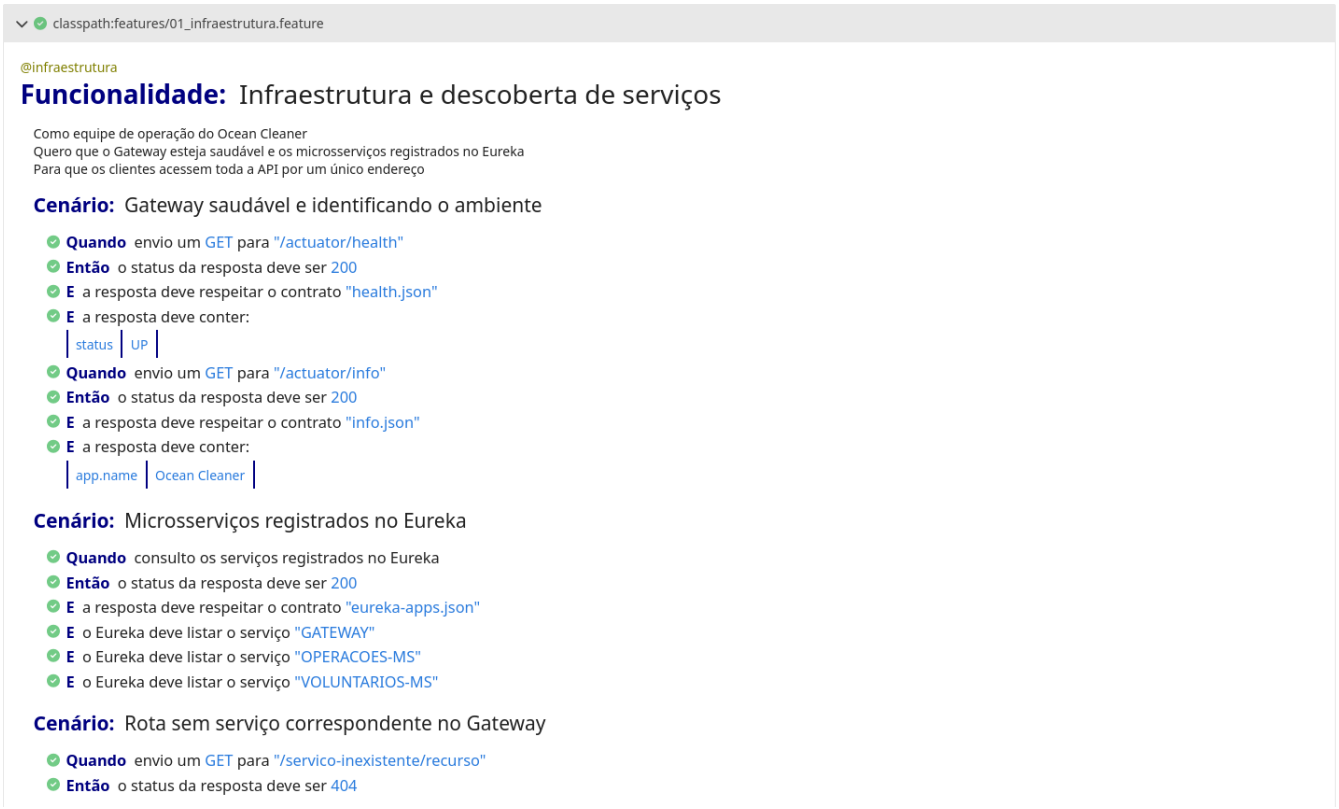


Figura 11 — Print da feature de infraestrutura no relatório HTML.

6.4 Relatório Cucumber — integração Feign

O detalhe do cenário de integração mostra os steps verdes, o corpo enviado e a validação do relatório criado no voluntarios-ms.



Figura 12 — Print do cenário de conclusão e relatório via Feign.

6.5 Rastreabilidade

O artifact e o run ficam disponíveis no GitHub Actions; o relatório HTML pode ser aberto diretamente e o XML é consumido pelo resumo do CI. Os dados de teste são removidos pelo hook ao final de cada cenário.

7. Pipeline de CI

O workflow encadeia build/testes unitários, testes de API E2E, imagens Docker e deploys. Em pull request, rodam o build/test e o job api-tests; o deploy fica para a master.

7.1 Etapas

- **build-test:** mvn verify nos quatro serviços, com JaCoCo e testes JUnit.
- **api-tests:** Docker Compose completo, 45 cenários Cucumber + Rest Assured, artifact api-tests-report.
- **docker:** imagens GHCR somente após build + api-tests, em push.
- **deploy staging/produção:** após imagens, com aprovação manual para produção.

7.2 Execução no PR

No PR #5, o job api-tests também foi executado e passou; os jobs de Docker e deploy ficaram como skipped, comportamento esperado para pull request. As evidências deste PDF usam a execução da master para refletir o código já integrado.

8. Como reproduzir

Pré-requisitos: Docker, Java 21 e um .env na raiz com as senhas. O Maven é baixado pelo wrapper da suíte.

8.1 Subir o ambiente e executar

```
cp .env.example .env
./api-tests/executar.sh --subir --derrubar
```

8.2 Contra ambiente já no ar

```
./api-tests/executar.sh
BASE_URL=http://localhost:8081 EUREKA_URL=http://localhost:8762 \
  API_PASSWORD=*** ./api-tests/executar.sh
```

8.3 Filtrar por tag

```
./api-tests/executar.sh -Dcucumber.filter.tags=@principal
./api-tests/executar.sh -Dcucumber.filter.tags=@integracao
```

8.4 Relatórios

O HTML, JSON e XML ficam em api-tests/target/. No CI, são publicados como artifact api-tests-report.

9. Conclusão, checklist e limites

A suíte do colega já cobre os requisitos da atividade. Não foi necessário alterar testes: a execução 41266518830 passou com 45/45 cenários. Este material registra a evidência dessa implementação.

9.1 Checklist da spec

Requisito	Situação	Evidência
Mínimo de 3 cenários Gherkin	Atendido	45 cenários, seção 3
Cenários positivos e negativos	Atendido	400/401/404/409/502
Testes para todas as APIs	Atendido	25 endpoints, seção 4.1
Validação de status code	Atendido	Todos os cenários
Validação do corpo JSON	Atendido	Tabelas de campos no Gherkin
Contratos JSON Schema	Atendido	13 schemas, seção 4.2
Comunicação entre serviços	Atendido	Feign real via Gateway; 502 e 409
Execução local/CI documentada	Atendido	Seção 8; execução no CI
Pipeline de CI/CD	Atendido	Job api-tests no workflow

9.2 Limites e observações

- **Dependência de ambiente:** a suíte exige Docker e Oracle; por isso as evidências usam a execução do GitHub Actions.
- **Escopo:** a suíte é E2E e cobre o caminho via Gateway; ela complementa os testes unitários/MockMvc já existentes.
- **Dados:** cada cenário cria e remove seus próprios dados, permitindo reexecução sem lixo no banco.
- **Diff de testes:** nenhuma alteração de teste foi necessária; o PR de evidências não modifica a suíte.

9.3 Conclusão

Os testes garantem a qualidade técnica das APIs e reforçam a contribuição ESG: o fluxo de resíduos é validado de ponta a ponta, os voluntários têm regras de integridade cobertas e o pipeline mantém rastreabilidade e bloqueio de regressões. O material permite demonstrar a entrega pela leitura dos 45 cenários e pelas evidências do CI.

Anexo A — Estrutura da suíte

```
api-tests/
├── executar.sh
├── pom.xml
└── src/test/
    ├── java/br/com/fiap/apitests/
    │   ├── RunCucumberTest.java
    │   ├── Ambiente.java
    │   ├── Api.java
    │   ├── Contexto.java
    │   └── steps/
    │       ├── ApiSteps.java
    │       ├── DadosSteps.java
    │       └── Hooks.java
    └── resources/
        ├── features/01_infraestrutura.feature
        ├── features/02_areas_maritimas.feature
        ├── features/03_voluntarios.feature
        ├── features/04_operacoes.feature
        ├── features/05_relatorios.feature
        └── schemas/*.json
```

Anexo B — Índice de evidências

Arquivo	Conteúdo
execucao-ci.png	Log real do job: 45/45 e BUILD SUCCESS
report-resumo.png	Relatório HTML: 45 PASSED / 100%
report-operacoes.png	Feature de operações expandida
report-integracao.png	Cenário de conclusão + relatório via Feign
config-*.png	POM, script, feature, schema, CI e README
logs/*.txt	Resumo do XML e log do CI

Documento gerado em 02/11/2026 • Ocean Cleaner • FIAP